

Security Audit

Libertum (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Libertum team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Libertum is a pioneering RWA tokenization platform that transforms real-world assets—real estate, art, commodities, and more—into ERC-20 tokens for fractional ownership and 24/7 on-chain liquidity on its licensed marketplace. Powered by the \$LBM governance and utility token (now listed on MEXC), Libertum leverages ERC-3643 security token standards and smart contracts to automate dividend distribution, staking, and community governance, eliminating intermediaries while enhancing transparency. With an intuitive dashboard, integrated DeFi modules for yield farming and borrowing, and a forthcoming Real-World Asset Bonding DEX in May 2025, Libertum democratically opens high-value markets to all investors and drives a community-powered circular economy.

Project Name: Libertum

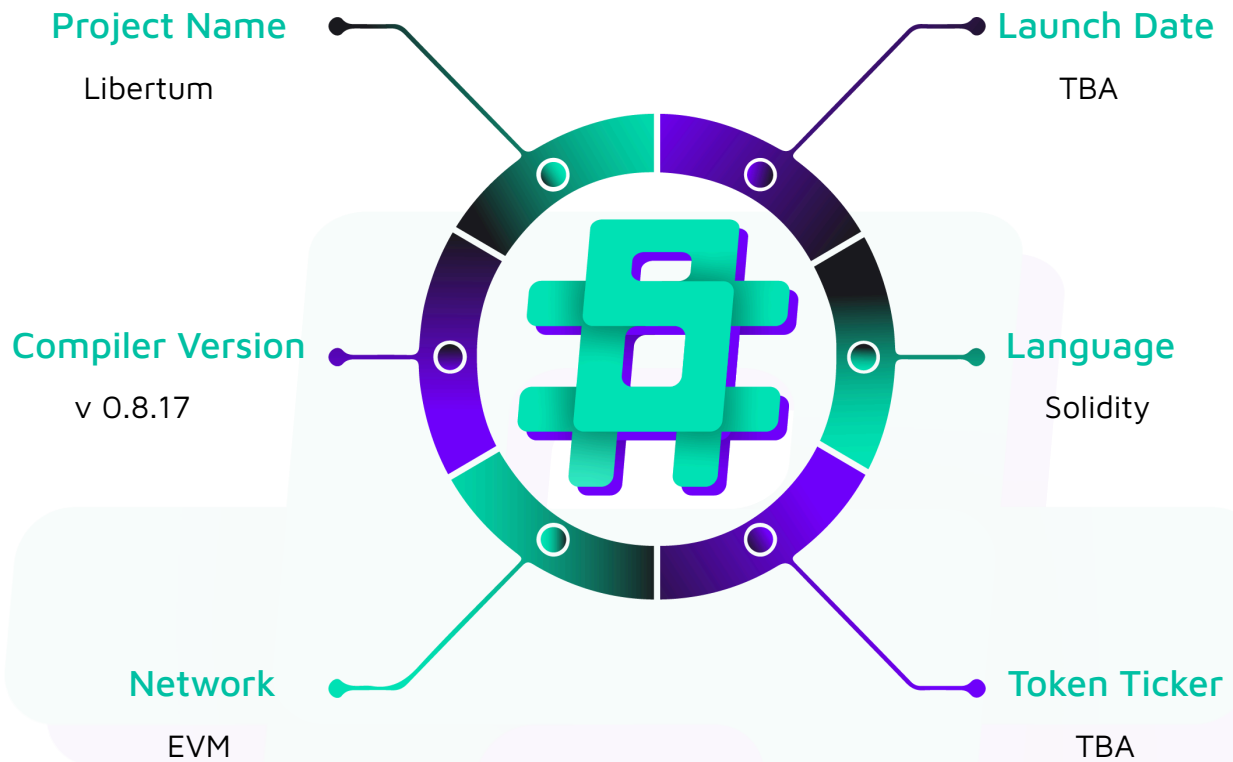
Project Type: Defi

Compiler Version: 0.8.17

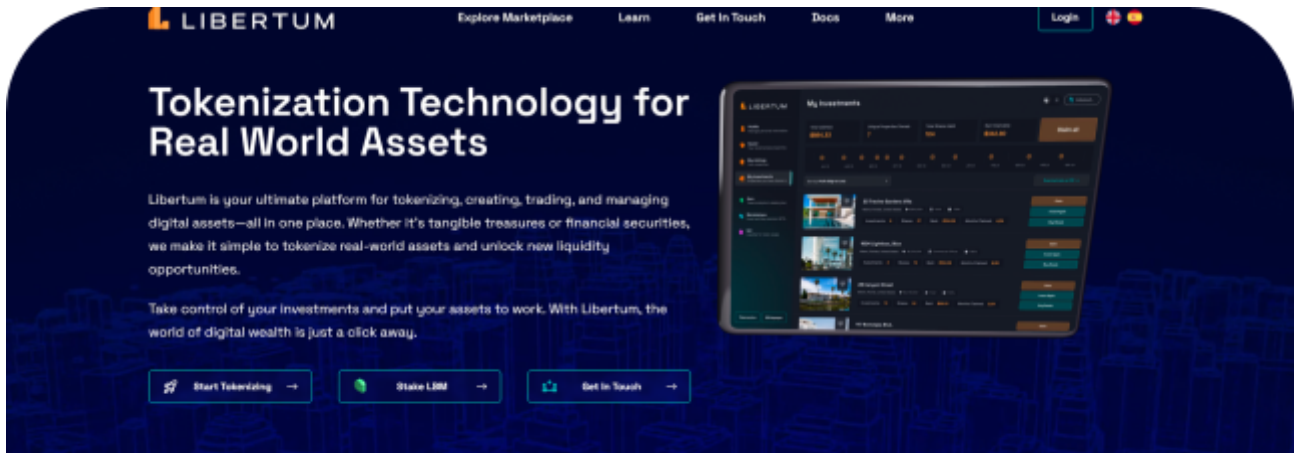
Website: <https://www.libertum.io/en/>

Logo:



Visualised Context:

Project Visuals:



Asset Owners

Access Interest-Free Capital for Your Next Investment

We are the next stage in the evolution of RWA: real estate, commodities, precious metals and gems, royalties, art, securities... Libertum will reduce the need for intermediaries, making RWA investment cost-effective and efficient.

🔥 Access to capital— interest-free, with terms you shape and define

🏠 Control Your Asset — unlike costly, complicated, and expensive alternatives, you maintain ownership of your property

⚡ Speed to market - List in weeks (not months), avoid traditionally expensive bureaucratic processes for real estate and other assets.

[Learn More →](#)

Audit Scope

We at Hashlock audited the solidity code within the Libertum project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Libertum Smart Contracts
Platform	EVM / Solidity
Audit Date	April, 2025
Audited Folder 1	contracts/compliance/modular
Audited Folder 2	contracts/compliance/modular/modules
Audited Folder 3	contracts/escrow
Audited Folder 4	contracts/factory
Audited Folder 5	contracts/fund
Audited Folder 6	contracts/onchainID
Audited Folder 7	contracts/onchainID/factory
Audited Folder 8	contracts/onchainID/proxy
Audited Folder 9	contracts/onchainID/storage
Audited Folder 10	contracts/onchainID/verifiers
Audited Folder 11	contracts/onchainID/version
Audited Folder 12	contracts/proxy
Audited Folder 13	contracts/proxy/authority
Audited Folder 14	contracts/registry/implementation
Audited Folder 15	contracts/registry/storage
Audited Folder 16	contracts/roles
Audited Folder 17	contracts/token

Audited GitHub Commit Hash	26d1b061d1198a2f44103c4639aa54cd3382dd91
Fix Review GitHub Commit Hash	f787200d3f8c78c1631c50de15b399ab10c46b9d

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

1 Medium severity vulnerability

3 Low severity vulnerabilities

2 Gas Optimisations

3 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ModularCompliance.sol - Allows owner to: - Bind/unbind tokens - Add/remove modules - Set/update a wrapper contract	Contract achieves this functionality.
EscrowController.sol - Allows users to: - Deposit stablecoins - Cancel orders - Allows owner to: - Rescue any ERC20 tokens - Set stablecoins - Set the master factory contract address - Set the fund factory contract address	Contract achieves this functionality.
FundFactory.sol - Allows owner to: - Set the implementation addresses for fund and equity configurations - Set the master factory contract address - Set fee parameters - Set admin wallets - Create a new fund contract - Create a new equity configuration contract	Contract achieves this functionality.
AgentRole.sol - Allows owner to: - Add/remove agents - Add/remove transfer agents	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Libertum project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Libertum project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] EscrowController#deposit - Incorrect handling of `totalPendingOrderAmount` value

Description

The `deposit` function in the smart contract allows users to deposit stablecoins, and the deposited amount is added to the `totalPendingOrderAmount` variable. However, the function does not account for the fact that different stablecoins may have varying decimal places (e.g., USDT has 6 decimals, while USDC has 6, and DAI has 18). This discrepancy in decimal precision leads to incorrect updates of the `totalPendingOrderAmount` variable, as the deposited amounts are not normalized to a consistent decimal standard.

Vulnerability Details

The `totalPendingOrderAmount` value is updated without normalization of amounts.

```
function deposit(address _token, uint256 _amount, uint256 _tokens, string calldata
orderId, string calldata coin) external {
    ...
    totalPendingOrderAmount += _amount;
    ...
}
```

Impact

- If a user deposits 1 USDT (6 decimals), it will be treated as 1,000,000 in the contract.
- If another user deposits 1 DAI (18 decimals), it will be treated as 1,000,000,000,000,000,000.

- Adding these values directly to `totalPendingOrderAmount` without normalization results in an inflated or incorrect total, which can lead to inaccurate calculations, mismanagement of funds, and potential vulnerabilities in the contract logic.

Recommendation

It's recommended to normalize all stablecoin deposits to a consistent decimal standard (e.g., 18 decimals) before updating the `totalPendingOrderAmount` variable.

Status

Resolved

Medium

[M-01] ModularCompliance, EscrowController, FundFactory, Fund, VERC20, Token - Anyone can initialise the implementation contracts

Description

The above contracts are upgradeable smart contracts.

The implementation contracts allow anyone to call the `initialize()` function due to the absence of a `_disableInitializers()` call in the constructor.

Impact

Anyone can take control of the implementation contracts.

Recommendation

It's recommended to add `oz-upgrades-unsafe-allow` constructor.

```
/// @custom:oz-upgrades-unsafe-allow constructor  
  
constructor() {  
  
    _disableInitializers();  
  
}
```

Status

Resolved

Low**[L-01] Contracts - Use Ownable2Step versions rather than Ownable versions****Description**

`Ownable2StepUpgradeable` and `Ownable2Step` prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` and `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

[L-02] FundFactory, TREXFactory - Important functions lack event emissions**Description**

Critical functions should emit events to provide transparency and enable off-chain monitoring of important state changes. Without events, it becomes difficult to track and verify contract operations, complicating both user interfaces and protocol monitoring. Instances are shown below.

```
function setAdminPatnerFeeDistribution() // FundFactory.sol
```



```
function setWrapper() // TREXFactory.sol
```

Recommendation

Review your contracts to implement event emissions for critical state variable updates.

Status

Resolved

[L-03] FundFactory, IdentityRegistry - Missing zero address validation

Description

The following functions do not include a validation check to ensure that the provided address parameters are not the zero address.

```
function init() // FundFactory.sol
function setImpl() // FundFactory.sol
function setMasterFactory() // FundFactory.sol
function setAdminWallets() // FundFactory.sol
function setIdentityRegistryStorage() // IdentityRegistry.sol
function setClaimTopicsRegistry() // IdentityRegistry.sol
function setTrustedIssuersRegistry() // IdentityRegistry.sol
```

Recommendation

Consider adding checks to ensure that the provided address parameters are not the zero address.

Status

Resolved

Gas

[G-01] EscrowController#settlement, cancelOrder - State variables that could be cached

Description

The `settlement` and `cancelOrder` functions read `order.value` value from storage multiple times.

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack.

Recommendation

Change the functions to cache the state variable that is used multiple times.

Status

Resolved

[G-02] FundFactory#setImpl, setMasterFactory - Could emit local variables

Description

The above functions emit the updated storage values at the end of the function.

However, the functions read the storage values to emit even if local variables with the same values exist.

Recommendation

Consider emitting the local variables.

Status

Resolved

QA

[Q-01] MaxBalanceModule - Unused error declaration

Description

The contract declares the `InvalidPresetValues` error which is never used in the contract.

Recommendation

Remove an unused error declaration.

Status

Resolved

[Q-02] ModularCompliance - Unused import

Description

The contract imports the `IToken` interface which is never used in the contract.

Recommendation

Remove an unused import.

Status

Resolved

[Q-03] EquityConfig#setMaxInvesrment - Typo in a function name**Description**

The function includes a typo in its name.

```
setMaxInvesrment() -> setMaxInvestment()
```

Recommendation

Fix the typo.

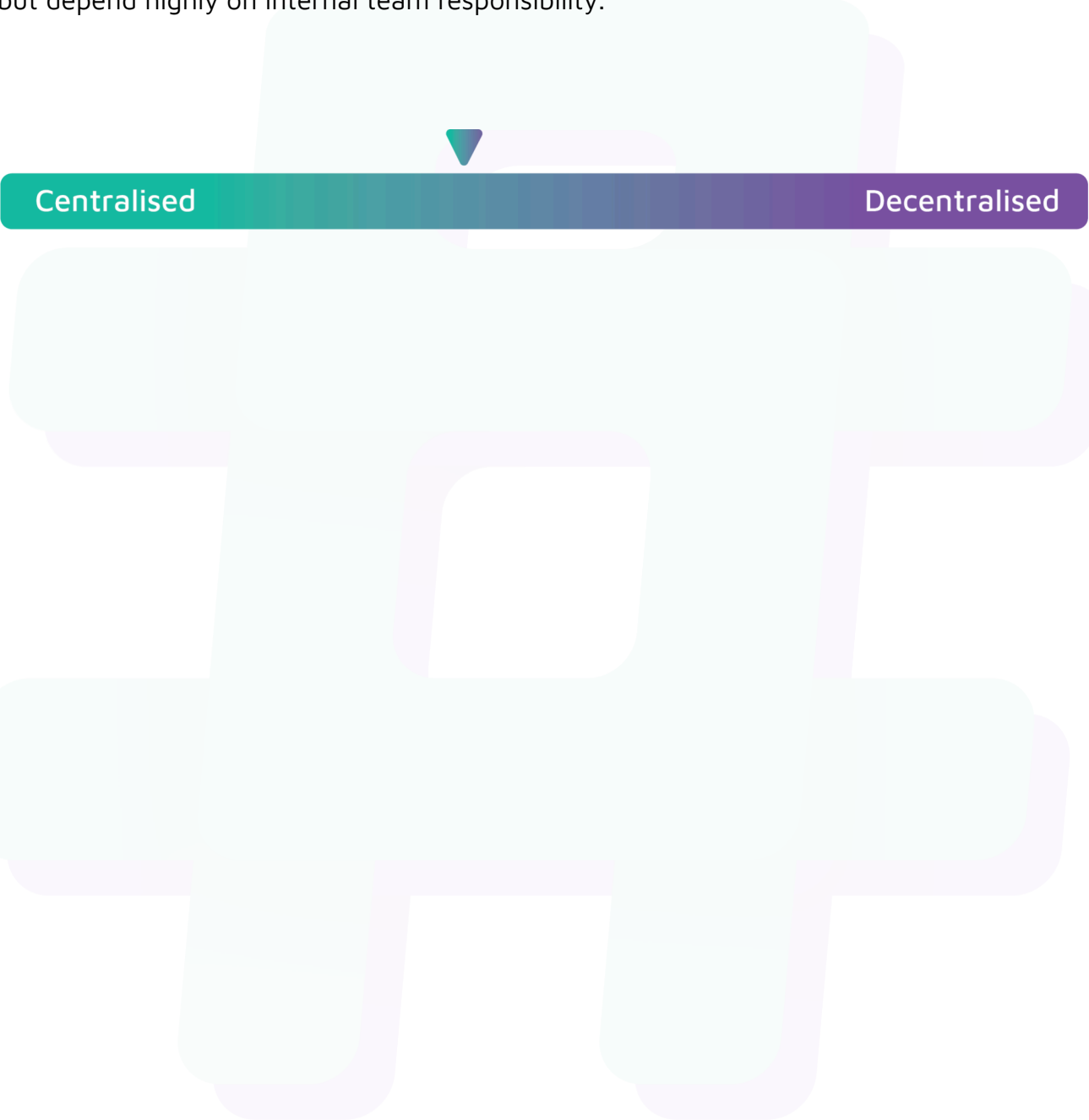
Status

Resolved

Centralisation

The Libertum project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Libertum project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.